Operating Systems (OS) — Answer Sheet

Name	Roll No
Rani Kumari	B25BS1270

Task 1 — Exploring the Virtual OS

The virtual environment was opened using copy.sh (v86), running an Android profile ("Android – v86") entirely inside the browser.

a. Operating system explored: Android, running as a browser-emulated virtual machine via the v86 emulator (copy.sh/v86, Android profile). The v86 control bar (Pause, Reset, Insert floppy image, etc.) belongs to the emulator itself, not to Android.

b. Four features/components observed:

- Home screen / desktop with a wallpaper background image.
- A Google search bar widget fixed at the top of the home screen.
- Application icons for Messaging, Dialer, Contacts, and Browser.
- A status bar at the top showing the system clock (9:15 AM) and notification/signal icons.

c. Part of the interface for files/folders: No dedicated file-manager icon is visible on this home screen. In a full Android build this role is normally played by a "Files"/"File Manager" app; here, apps such as Messaging and Contacts manage their own data internally, and the Browser can access downloaded files. So, strictly from what is visible, the interface does not expose an explicit files/folders view.

d. Part used to execute commands: The serial console at the bottom of the page ("This is the serial console. Whatever you type or paste here will be sent to COM1"). This is a text console provided by the v86 emulator for sending low-level commands/output to the virtual machine.

Screenshot of the virtual OS is attached at the end of this document.

Task 2 — Virtual OS vs. Google Colab

Comparison table:

Feature	Virtual OS on copy.sh	Google Colab Environment
Graphical desktop available	Yes (full Android home screen/GUI)	No (only a notebook/code-cell interface)
Can run commands	Yes (via the serial console / within the guest OS)	Yes (Python code cells; shell commands with "!")
Can create files	Yes (within the emulated Android filesystem)	Yes (within the temporary Colab VM filesystem)
Can run Python programs	Not natively (Android is not set up for general Python execution)	Yes (native Python/Jupyter runtime)
User directly controls the complete OS	Yes (full virtual machine, boots an entire OS)	No (a managed, sandboxed cloud VM behind a notebook interface)

a. Two similarities: (1) Both run entirely inside a web browser without installing anything on the local machine. (2) Both let the user execute commands/code and create files inside a virtual environment.

b. Two differences: (1) The virtual OS provides a full graphical desktop and boots a complete operating system, while Colab only exposes a notebook of code cells with no desktop. (2) Colab is built for running Python directly with pre-installed libraries, while the Android VM is not designed for general-purpose Python execution.

c. Why Colab doesn't behave like a normal OS: Colab gives access to only one interface (the notebook) into a temporary, sandboxed cloud virtual machine. There is no desktop, no direct hardware access, the session resets and loses its files after

disconnection/inactivity, and resources (CPU, RAM, runtime) are shared and time-limited by Google. It is designed purely for running code, not for general end-user computing.

Task 3 — Programs vs. Processes

a. Applications that become a process when executed: All four — Web browser, Music player, Text editor, and Calculator — become a process the moment they are launched and are running in memory.

b. Program vs. process: A program is a passive set of instructions stored on disk (e.g., an .exe or app file) that does nothing by itself. A process is that program loaded into memory and actively executing — it has its own PID, memory space, CPU state, and resources allocated by the OS.

c. Opening the browser three times: Yes. The operating system can manage multiple processes at once. Each launch creates a separate process (with its own PID), and the OS scheduler shares CPU time among all of them, giving the appearance that they run simultaneously.

d. Why the OS must track running processes: It needs to allocate CPU time, memory, and other resources fairly; keep each process's data isolated and protected from others; know each process's current state (running/ready/waiting) so it can schedule correctly; and be able to suspend, resume, or terminate processes cleanly without corrupting the system.

Task 4 — Viewing Running Processes with psutil

Python code (to run in a Google Colab cell):

```
!pip install psutil

 import psutil

for proc in list(psutil.process_iter(['pid', 'name']))[:5]:
print(proc.info['pid'], proc.info['name'])
```

Sample recorded output (five running processes):

PID	Process Name
1	process_api
2	kthreadd
3	pool_workqueue_release
4	kworker/R-rcu_gp
5	kworker/R-sync_wq

Note: exact PIDs and process names will differ each time the code is run, since Colab assigns a fresh virtual machine per session.

a. What PID represents: PID (Process ID) is a unique numeric identifier the operating system assigns to a process while it is running, used to reference, monitor, and control that specific process.

b. Why every process needs an identifier: So the OS (and the user) can uniquely distinguish between many simultaneously running processes — even multiple instances of the same program — in order to allocate resources, send signals, monitor status, and terminate the correct one without affecting others.

Task 5 — Adding Process Status

Modified Python code:

```
import psutil

for proc in list(psutil.process_iter(['pid', 'name', 'status']))[:5]:
print(proc.info['pid'], proc.info['name'], proc.info['status'])
```

Sample recorded output:

PID	Process Name	Status
1	process_api	sleeping
2	kthreadd	sleeping
3	pool_workqueue_release	sleeping
4	kworker/R-rcu_gp	idle
5	kworker/R-sync_wq	idle

a. Did all processes have the same status? No — in the sample output some processes were "sleeping" while others were "idle"; statuses are not identical across processes.

b. What the process status tells us: It tells us what a process is currently doing/able to do — e.g., actively running, sleeping/waiting for an event, idle, or stopped — which reflects its position in the process lifecycle at that instant.

c. Why status might change while the system is running: Processes constantly move between states as the OS scheduler allocates CPU time, as processes request or complete I/O operations, as they wait for resources or user input, or as they finish execution — so a status captured at one moment can differ moments later.

Task 6 — Identifying Process States

a. Program currently performing calculations on the CPU: Running

b. Program ready to run but another process is using the CPU: Ready

c. Program waiting for user input: Waiting/Blocked

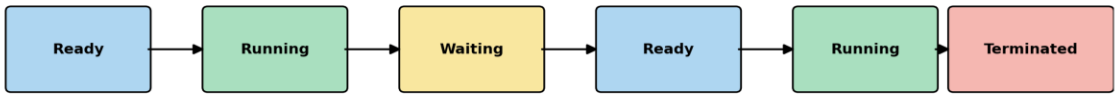
d. Program that has completed execution: Terminated

e. Program waiting for data to be read from a file: Waiting/Blocked

Why a process may move from Running to Waiting: A running process moves to the Waiting state when it needs something it cannot get immediately — such as data from disk, a network response, or user input. Since the CPU would otherwise sit idle while the process waits, the OS takes the process off the CPU and places it in Waiting, freeing the CPU for other Ready processes until the required event/resource becomes available.

Task 7 — Process State Diagram

Diagram showing: Ready → Running → Waiting → Ready → Running → Terminated



a. What causes a process to move from Ready to Running? The CPU scheduler (dispatcher) selects it from the ready queue — usually because the CPU becomes free or the process's turn/priority comes up — and assigns the CPU to it.

b. Why can a running process move into the Waiting state? Because it needs to pause for an event or resource that is not immediately available, such as I/O completion, a lock, or user input, and cannot keep using the CPU productively until that happens.

c. After the resource becomes available, which state can the process move to? It moves to the Ready state (it must wait its turn for the CPU again before it can resume Running).

d. Which OS component decides which ready process gets CPU time? The CPU scheduler (short-term scheduler/dispatcher), part of process/CPU management in the operating system.

Task 8 — Multiple Applications, Limited Resources

Response (6–8 sentences):

Not all of these applications truly use the CPU or memory at the exact same instant, even though they appear to run simultaneously. Modern operating systems use multitasking: the CPU rapidly switches between the browser, video player, Python program, file download, and text editor, giving each a small time slice, so from a human perspective everything seems to happen at once. The operating system manages this by keeping every application as a separate process, tracking each one's state (running, ready, or waiting), and using a scheduler to decide which process gets the CPU next. If one process — say the Python program — consumes too much CPU or memory, the other applications can slow down, become unresponsive, or in extreme cases the system may run out of memory and be forced to terminate a process. This is exactly why process monitoring is useful: by watching CPU and memory usage per process, a user or the OS itself can identify which application is causing performance problems and act on it, for example by closing or throttling it. If the file download is waiting for network data, that process enters the Waiting/Blocked state until the data arrives, freeing the CPU for the other applications in the meantime. Overall, process management — creating, scheduling, monitoring, and terminating processes — is one of the operating system's most important responsibilities, because it is what allows a computer with limited CPU and memory to run many applications smoothly, fairly, and safely at the same time.

Appendix — Screenshot of the Virtual OS (Task 1)

